



Dissertation on

“AVIOS – Adaptable Vigilant Intelligent Operating System”

*Submitted in partial fulfillment of the requirements for the award of the degree
of*

**Bachelor of Technology
in
Computer Science & Engineering**

UE22CS320A – Capstone Project Phase - 1

Submitted by:

KURUMETI PRAJVAL	PES2UG22CS274
MANOJ KUMAR KS	PES2UG22CS302
DHANUSHREE K	PES2UG22CS176
NAGARATNA	PES2UG22CS331

Under the guidance of

Prof. Swetha Patil

Designation
PES University

Aug-Dec 2024

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PES UNIVERSITY**

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India



PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

FACULTY OF ENGINEERING

CERTIFICATE

This is to certify that the dissertation entitled

‘AVIOS – Adaptable Vigilant Intelligent Operating System’

is a bonafide work carried out by

**KURUMETI PRAJVAL
MANOJ KUMAR KS
DHANUSHREE K
NAGARATNA**

**PES2UG22CS274
PES2UG22CS302
PES2UG22CS176
PES2UG22CS331**

In partial fulfillment for the completion of Fifth-semester Capstone Project Phase - 1 (UE22CS320A) in the Program of Study -Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period Aug 2024 – Dec. 2024. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report. The dissertation has been approved as it satisfies the 5th-semester academic requirements in respect of project work.

Signature
Prof. Swetha Patil
Assistant Professor

Signature
Dr. Sandesh B J
Chairperson
External Viva

Signature
Dr. B K Keshavan
Dean of Faculty

Name of the Examiners

Signature with Date

1. _____

2. _____

DECLARATION

We hereby declare that the Capstone Project Phase - 1 entitled “**AVIOS – Adapative Vigilant Intelligent Operating System**” has been carried out by us under the guidance of **Prof. Swetha Patil, Assistant Professor** and submitted in partial fulfillment of the course requirements for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester Aug. – Dec 2024. The matter embodied in this report has not been submitted to any other university or institution for the award of any degree.

PES2UG22CS274
PES2UG22CS302
PES2UG22CS176
PES2UG22CS331

KURUMETI PRAJVAL
MANOJ KUMAR KS
DHANUSHREE K
NAGARATNA

ACKNOWLEDGEMENT

I would like to express my gratitude to **Prof. Swetha Patil**, Department of Computer Science and Engineering, PES University, for his/ her continuous guidance, assistance, and encouragement throughout the development of this UE22CS320A - Capstone Project Phase – 1.

I am grateful to all Capstone Project Coordinators, for organizing, managing, and helping with the entire process.

I take this opportunity to thank Dr. Sandesh B J, Professor & Chairperson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department. I would like to thank Dr. B.K. Keshavan, Dean of Faculty, PES University for his help.

I am deeply grateful to Dr. M. R. Doreswamy, Chancellor, PES University, Prof. Jawahar Doreswamy, Pro-Chancellor, PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University, and Prof. Nagarjuna Sadineni, Pro-Vice Chancellor, PES University, for providing me with various opportunities and enlightenment every step of the way. Finally, Phase 1 of the project could not have been completed without the continual support and encouragement I have received from my family and friends.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	01
2.	PROBLEM STATEMENT	03
3.	ABSTRACT AND SCOPE	05
4.	RESEARCH / TECHNOLOGY GAP AND CHALLENGES	06
5.	OBJECTIVES	07
6.	LITERATURE SURVEY	13
	6.1 Development of an Operating System Supporting Intelligent Predictions and Recommendations	
	6.2 A Methodology for Soft Errors Detection And Automatic Recovery.	
	6.3 AI based OS: Future of Operating System	
	6.4 An Overview of Fault Tolerance Techniques for Real-Time Operating Systems	
	6.5 Android API Malware Detection based on Permission Using Machine Learning and Deep Learning Methods	
	6.6 Application of Expert Systems in Quality Management	
	6.7 Artificially Intelligent Operating System with Sapi5 Voice Recognition Engine	
	6.8 OS Agents: Using AI Techniques in the Operating System Environment	
7.	Overview of Datasets	17
	7.1 Description	
	7.2 Data Attributes / Features	
8.	CONCLUSION OF CAPSTONE PROJECT PHASE – 1	18
9.	PLAN OF WORK FOR CAPSTONE PROJECT PHASE – 2	19

REFERENCES/BIBLIOGRAPHY

APPENDIX A DEFINITIONS, ACRONYMS, AND ABBREVIATIONS

CHAPTER 1: INTRODUCTION

In this modern era, Operating systems have become the core of our daily competitive experience. Linux, Has One of the most widely used operating system, continuous to evolve rapidly to meet the increasingly complex needs of users[1]. One of the main challenges user faces is how to increase productivity and efficiency in using the applications within the Operating System [2][3]. Although Linux offers high flexibility and control, users offers encounter difficulties in finding and using the applications that suits for their needs.[4].

Artificial Intelligence operating system is the transformation step in Operating system. Where we integrate AI technologies to enable intelligent automation productive analysis and increase system Performance. Unlike Traditional OS architecture, AI OS try to identify patterns, context- aware processing and personalization to deliver a highly responsive and adaptive computer environment. These are designed to Operate across diverse platform like Personal computers, IOT devices and enterprise network make them versatile and future ready.

AIOS aims to address these challenges by developing a intelligent operating system capable of providing applications recommendations based on individual usage patterns and performance. The system will use artificial intelligence technology to analyze applications usage data and user behavior pattern. [5][6]. Thus, it is expected that this system can help Linux users choose and use applications more effectively, accordingly to their daily needs.

Advancements in distributed Operating systems are coming up exciting possibilities for adding AI to OS design. Distributed system use multiple processors, memory and large storage devices and make them powerful enough to run complex AI tasks. For example, applications like distributed knowledge networks, distributed natural language summarization, and parallel such using machine learning, so how this systems can improve decision making, information retrieval and data analysis. These Innovations create a solid base for developing smarter, AI powered Operating Systems in the Future.

The Evolution of Operating systems, from IBM mainframes to modern platforms like Linux and MacOS highlights the potential for AI to transform how Operating system functions. AI based operating system bring advanced features such as predictive intelligence context awareness search, associative reasoning and intelligent memory and storage management. These capabilities allow AIOS to take care of complex problems with great efficiency. For example in systems like windows metro with the help of AI integrations for personalized assistance and predictive task management. AIOS consists consider these ideas further and provides more powerful, scalable, efficient systems.

This study aims to integrate the power of AI with the flexibility of Linux by creating a adaptive, intelligent operating system. The goal is to improve application recommendations optimize resource use, and system operations by identifying challenges application discovery and management this not only tackles immediate user needs but also pushes forward the integration of AI in open source systems. The aim is to built smart and more responsive computing environment that aligns with the future of operating systems design and functionalities.

CHAPTER 2: PROBLEM STATEMENT

Integrating AI into operating system brings excellent opportunities but also challenging traditional operating system are built to handle fixed tasks and static user needs. They lack the adaptability and intelligence to respond dynamically changing user behaviors, Changing application demands and evolving hardware setups. This becomes more evident as computing environments to grow more complex and diverse, Ranging from personal devices and IoT platforms to large scale systems.

In today's computing world user expect operating system to do more than manage resources efficiently, they want systems that can anticipate their needs, boost productivity performance and simplify interactions. However, most OS architecture doesn't have built in mechanism intelligent decision making and automation. While some systems have introduced basic AI driven features like Voice recognition and task prediction, these implementations are often limited by scope and functionalities.

Benefits of Integrating AI into OS :

- **Personalized Application Recommendations :** AI can analyze user behavior and usage patterns to suggest apps to the individual needs which saves time improves experience.
- **Resource Management :** AI can help in dynamic allocation of system resources based on the work loads and increases efficiency and overall system performance.
- **Enhance Security :** AI can detect and respond to security threats in real time by identifying unusual behaviors and improving system safety.
- **Automation of Routine Tasks :** With the help of Machine learning algorithms, AI can automate repetitive tasks allowing user to focus on more meaningful activities.
- **Adaptive system behavior :** AI integrated Operating system can evolve overtime by learning from user interpretations to better meet changing needs and application demands.

CHAPTER 3: ABSTRACT AND SCOPE

Operating system is the backbone of computational system we will see how integration of in OS makes the systems smarter and adoptable. Traditional operating systems manage hardware and software well but they don't have the intelligence to improve the user experience or make operations more efficient on their own with the help AIOS we can improve user experience by context aware search , personalized recommendations, intelligent allocations and predictive system maintenance. Using techniques such as neural networks pattern recognition and fuzzy logic, AI OS aims to manage system management and saves user time and also improve overall performance.

In this we are mainly focusing on Linux systems aim to improve common challenges, such as finding and using the right applications AI and AI powered recommendation systems that looks at how applications are used user behavior to provide personalized suggestions. This systems use python based tools like psutil, pandas, scikit-learn, joblib to process data and train machine learning models. These results shows a significant boost in user productivity and efficiency. With the help of Machine learning algorithms, reinforcement learning and supervised based learning for making predictions and recommendations for the user. By using distributed system architecture, AI driven systems can manage complex tasks more efficiently scale across different environments.

This study concentrate on the greater potential of integrating AI into operating system by connecting the gap between the capabilities of traditional OS and the need of modern computing AIOS provides a smarter more efficient, and user friendly platform. This research aims in identifying challenges like finding the right applications and also shows the way for next generations systems that are more responsive, secure and adaptable

CHAPTRE 4: RESEARCH / TECHNOLOGY GAP AND CHALLENGES

Research Gap

1. **Limited AI Models for Fault Detection:**

While AI has been applied in many fields, its application to fault detection in operating systems remains underexplored. The lack of robust, adaptive AI models capable of identifying and responding to faults autonomously is a significant gap.

2. **Insufficient Datasets:**

There is a scarcity of datasets that represent diverse fault scenarios in operating systems, limiting the effectiveness of training machine learning models for accurate fault prediction and recovery.

3. **Scalability Challenges:**

Current fault detection systems often fail to scale effectively in memory-intensive or high-load environments, leading to performance bottlenecks.

4. **Integration Challenges:**

Integrating fault detection mechanisms with self-healing capabilities while maintaining system stability is a complex and underexplored area in OS design.

5. **Real-Time Optimization:**

Existing solutions lack the efficiency needed to operate in real-time without introducing significant computational overhead, especially during fault recovery processes.

Challenges and Solutions in Implementation

1. User Data Management:

Challenge: Large-scale data collection for fault analysis can strain system storage and processing capabilities.

Solution: Employ data compression techniques and distributed storage systems to minimize storage needs. Parallel data processing algorithms can further enhance efficiency.

2. User Privacy:

Challenge: Fault detection systems require access to system-level data, raising concerns about user privacy and data security.

Solution: Implement robust data encryption protocols and provide transparency about data usage. Allow users to control what data is collected and used.

3. System Performance:

Challenge: Running AI models for fault detection and self-healing can impact OS performance, especially on low-spec devices.

Solution: Optimize AI algorithms using techniques like model distillation to reduce computational demands. Leverage hardware acceleration, such as GPUs or TPUs, to handle complex processing.

4. Integration with Existing Systems:

Challenge: Integrating fault detection and self-healing mechanisms into the Linux kernel can create conflicts with existing functionalities.

Solution: Conduct extensive compatibility testing across various hardware and software configurations. Containerization can isolate new AI features to ensure system stability.

5. Resource Availability:

Challenge: Developing and maintaining advanced fault detection features requires significant expertise, time, and funding.

Solution: Adopt open-source collaboration to share development efforts. Engaging with the open-source community can accelerate progress and improve the system's quality.

CHAPTER 5: OBJECTIVES

The following are the objectives of our project AVIOS:

- **To Enable Autonomous Fault Detection:**

Design AI algorithms capable of identifying system faults, such as memory leaks or hardware failures, without user intervention.

- **To Implement Self-Healing Mechanisms:**

Develop automated recovery systems that diagnose, isolate, and resolve faults dynamically to maintain system uptime and reliability.

- **To Enhance System Security:**

Incorporate AI-based threat detection to monitor real-time system behavior and mitigate vulnerabilities proactively.

- **To Improve Memory Allocation During Fault Recovery:**

Create optimized memory management techniques that function efficiently during self-healing processes, ensuring seamless performance.

- **To Adapt to Diverse System Environments:**

Ensure the solution works effectively across different hardware and software configurations, maintaining compatibility and scalability.

- **To Deliver Scalable and Efficient Systems:**

Minimize computational overhead by optimizing AI-driven fault detection and recovery systems for real-time operations.

CHAPTER 6: LITERATURE SURVEY

6.1 Development of an Operating System Supporting Intelligent Predictions and Recommendations

6.1.1 Introduction

In the modern era, Operating System have become a core computational part in our day to day life. User often are still not satisfied by the existing Operating System user's want automation like task automation, Self care of the system and many more features. Linux one of the most well know Operating System in the world rapidly evolving to meet user's complex needs. Even though Linux offers the most efficient and feasible Operating system environment user often encounter difficulties in the applications that best suit them.

This Research aims to provide a intelligent Operating System that suits user need and move forward with the help of evolving era of Artificial Intelligence and application usage and user behavior pattern and predict results accordingly.

6.1.2 Characteristics and Implementation

1. User Data Management

Implementation: One of the techniques to reduce load over the system is to use distributed storage system.

2. User Privacy

Implementation: Implement Strict encryption and security protocols to maintain data security. Additionally, we can give user control to manage data collected.

3. System Performance

Implementation: Use hardware like GPU or TPU can help has run the AI algorithms very smoothly without any disturbances.

6.1.3 Features

Main feature of the research is to bring a effective change in the Operating System where user doesn't need to manually search for a application or search for a service. We want to add a mechanism that behaves like the user and give recommendations on the basis of user application or service usage patterns which makes the Operating system smarter and more efficient.

6.1.4 Evaluation

Benefits

1. **Increased User Productivity:** By providing recommended Applications or services we can reduce the time taken to search very drastically by improving user throughput in a significant manner.
2. **More Efficient Resource Usage:** When we evaluate the resource usage needed before hand according tot user usage patterns making it less prone to potential error during the allocation.

Future Developments

1. *Integrating IoT Devices:* Integrating to IoT make our system more user preferred environment making it smarter like user preference light setting and many more. IoT integration can change the whole game of manual mode switch according tot user preference. Even tho there are system in the market which make modes how user wants we can make a enhancement of according to user mood we can set the layout.
2. *Enhanced Security:* We can integrate Security with AI Algorithms making it more reliable and more effect mitigation wrt to the problem predicted.

6.2 A Methodology for Soft Errors Detection and Automatic Recovery

6.2.1 Introduction

System reliability is one of the most crucial part of a High-Performance Computing (HPU) devices. On high performance task deeply rely on. Silent Data Corruption (SDC), where applications produce invalid results despite appearing correct, is particularly dangerous for scientific simulations, which rely on precise computations.

Traditional fault tolerance methods like memory replication and checkpoint-based rollback recovery methods provide partial solutions but not the complete solution. However, efficiency decreases as cores increase.

6.2.2 Characteristics and Implementation

1. Error Detection with notification

Implementation: Using process duplication, it is capable of performing only detection and help in voting process.

2. Recovery based in multiple system-level checkpoints

Implementation: There are two cases possible

- Fault occurred and detected before next checkpoint: We can restart the system or application from the previous checkpoint.
- Fault occurred before checkpoint was created: We can restart the system or application but we can't restart the system from last checkpoint which is not possible. So, restart should be done from prior checkpoint.

6.2.3 Features

Majorly we focus on detecting and mitigating error in the system in an efficient manner and reliable manner.

6.2.4 Evaluation

We evaluate key finding like overheads, fault tolerance gains, multiple checkpoints, and application specific impact. There are tests done in this particular paper the below Fig 6.2.4.1 represent the test results.

Situation	Execution Time [hs]
Baseline, without fault (Eq. 1)	11
Baseline, with fault (Eq. 2)	22.3
Only detection, without fault (Eq. 3)	12.5
Only detection, with fault (Eq. 4)	18.5
Multiple checkpoints, without fault (Eq. 5)	16.1
Multiple checkpoints, with fault (Eq. 6, $k = 0$)	17.7
Multiple checkpoints, with fault (Eq. 6, $k = 1$)	22.6
Single checkpoint, without fault (Eq. 7)	18.5
Single checkpoint, with fault (Eq. 8)	20

Fig 6.2.4.1 shows multiple scenarios testing of the system

6.3 AI based OS: Future of Operating System

6.3.1 Introduction

An Artificial Intelligence Operating System (AIOS) is a system that manages System Hardware and software in a efficient manner via Artificial Intelligence. This Paper talks how we can integrate Artificial Intelligence into the daily needed computational device making it smarter and efficient. Integrating AI, we can drastically cut down time in process management, memory management, and improved security.

6.3.2 Characteristics and Implementation

Characteristics: Artificial Intelligence Operating System (AIOS) has capabilities like Smart memory and process management, Enhances Security, Self-Healing Capability to recover from Cyber Attacks. With these features System optimizes system performance and efficiency.

Implementation: AIOS relies on Strong AI components like Fuzzy Logic for decision making when uncertainty, expert Systems for human like problem solving, and neural networks for learning user behavior and usage patterns. Models created will ensure that they allocate resource more proactively to reduce latency. AIOS gives scalable feature to various platforms making more use out of it.

6.3.3 Features

1. Smart Process Management
2. Smart Memory Management
3. Enhances Security
4. Predictive maintenance
5. Deriving solution for a problem on its own
6. Healing from cyber attacks

6.3.4 Evaluation

The Biggest advantages of AIOS is it can learn continuously and grows wrt to time. As Human learns from his/her mistakes this system also learns from its error between the predicted output and the actual output that the user requires.

The disadvantages of AIOS making systems more automated Humans tend to become lazier which is one harmful impact on the future generations. We as user should not completely rely on the AI and its advancements, we should keep in mind that AI will not predict actual result with 100% accuracy its gives most predicted result that user might expect.

6.4 An Overview of Fault Tolerance Techniques for Real-Time Operating Systems

6.4.1 Introduction

This paper majorly discusses the techniques for tolerating faults for RTOS. Faults are one of the major problems in the modern existing systems. So, we can encounter the problem using multiple techniques proposed by the Research paper. Few Techniques are by replicating the usage memory for that process and use voting system to get the right schedule of process.

6.4.2 Characteristics and Implementation

RTOS majorly prioritizes reliability and error tolerance. Key features include robust Memory management, precise task scheduling, and error recovery mechanism from faults occurring in the system.

Implementation: RTOS uses techniques like memory duplications to safe guard data, timers to meet the real time deadlines, and error logging to find the root cause. Schedule resilience is achieved by replication and voting process. These methods increase reliability and efficiency.

6.4.3 Features

1. *Memory Management:* As flexibility and functionality increase, we need applications to have dynamic allocation strategies for making more out of the system.
2. *Kernal Considerations:* Whenever there are issues in the hardware or software the fault RTOS should notify the agent and start taking necessary actions by isolating a address space specially only for fault tolerance resolving section. Why isolation because it has all the fault process which will resolved without causing inconvenience to the current running process.
3. *Scheduling:* Scheduler is most protected part of fault RTOS against the failures. If scheduler fails all task come to halt and not function any further. Fault RTOS uses a special type of scheduler making itself different from the existing scheduler called N-Copy programming (NCP). This NCP typically creates copies of the process and runs all the copies in different address space locations and correct scheduling is done with voting system from the replicas.

6.4.4 Evaluation

Evaluation can be done by introducing faults into the system and check if it works. Fault RTOS can lead to major impact in the future making system more reliable.

6.5 Android API Malware Detection based on Permission Using Machine Learning and Deep Learning Methods

6.5.1 Introduction

Major concerning issue in today's world it to be protected from the malware attacks done on various operating systems throughout the world one of the most targeted operating system is Android. Because Android is one the most used and easy target to attackers if the User has no proper knowledge about the cyber-attacks and permissions given to applications and services.

Android malware can be detected with the help of network analysis, API and permissions approach. Android permission feature allows user to decide which application can access and use sensitive user data like camera, microphone and many more.

6.5.2 Characteristics and Implementation

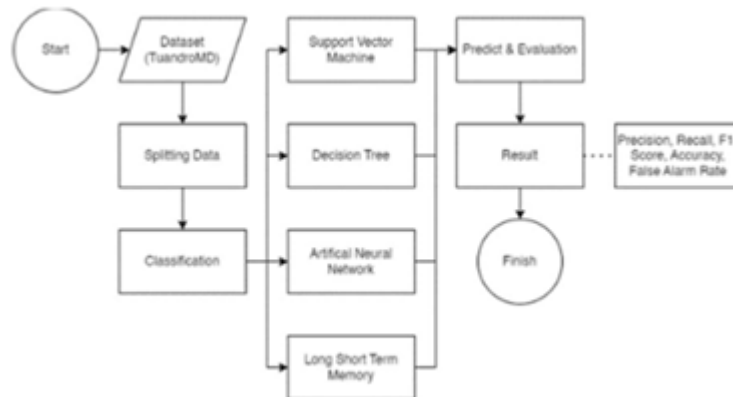


Fig 6.5.2.1 Flow chart of the System.

1. *Dataset Description:* Dataset with malware detection and permission requested for android application.
2. *Binarization:* Conversion of existing data into 0 or 1.
3. *Splitting Data:* 25% Training data and 75% Testing data.

6.5.3 Features

Major feature is to implement Artificial Intelligence Algorithms for effective detection and protection against the malware accessing the sensitive data in your device.

6.5.4 Evaluation

We can use various Machine learning techniques for effective implementation. Below Figure 6.5.4.1 shows the performance metrics conducted by Research paper *“Android API Malware Detection based on Permission Using Machine Learning and Deep Learning Methods”*[12].

Method	Value				
	Precision	Recall	F1-Score	Accuracy	False Alarm Rate
SVM	89%	90%	90%	93%	14.34%
DT	98%	97%	98%	99%	4.78%
ANN	99%	99%	99%	99%	1.3%
LSTM	89%	90%	90%	93%	17.83%

Fig 6.5.4.1 Performance metric

6.6 Application of Expert Systems in Quality Management

6.6.1 Introduction

As the Information Technology industry is every growing which allow us to bring new tech and require high restructuring cost and quality management system tools (QMS) that the implementation of tech is more reliable and feasible.

As the Artificial Intelligence field grows expert systems has come into lime light especially after integrating Artificial Intelligent making it more efficient and better performing.

6.6.2 Characteristics and Implementation

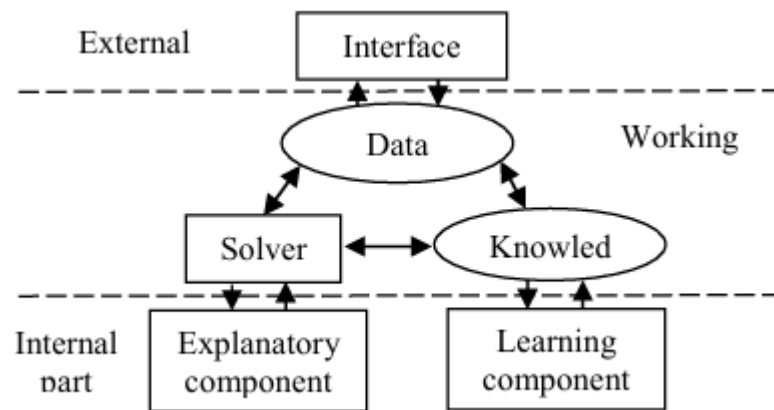


Fig 6.6.2.1 Expert System Architecture

External Part contains Interface which is mode of communication between User and the expert system.

Working Part contains Data, Knowledge base, and solver. Data comes from the external part, knowledge database part contains the previous knowledge and data collected from the system, Solver computes the predicted result and communicates with the internal part.

Internal Part contain two components Explanatory component and Learning component. The Learning part main use it to keep track of knowledge part and learn automatically from knowledge acquired, and Explanatory component main purpose to make a detailed solution and give it the system.

Implementation: Using Artificial Intelligence algorithms, we can effectively implement Expert systems making it more reliable while quality management.

6.6.3 Features

1. Interface
2. Data base
3. Knowledge base
4. Solver
5. Explanatory component
6. Learning component

6.7 Artificially Intelligent Operating System with Sapi5 Voice Recognition Engine

6.7.1 Introduction

As technologies have increased their limits, we have seen many technology endorsements in the recent year one of the major feature making our life more easier is the voice technology. Today's tech is very easy but just using voice commands to make some operations.

Voice technology has a major impact toward the disabled people which is a major alternative to using keyboard, mouse and conventional methods of human interaction with the system.

6.7.2 Characteristics and Implementation

1. *Speech Recognition Engine* used to recognize the input data from user. System should take the command first from user, process the instruction and then do the task. Fig 6.1.2.1 shows how the speech recognition engine works.

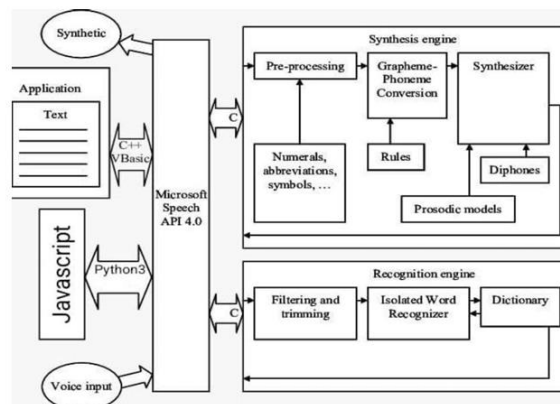


Fig 6.7.2.1 speech recognition engine working

Implementation

1. Acoustic models
2. Language models

6.7.3 Features

1. Text to Speech

Using pyttsx3 python library we can convert Text to Speech. The below Fig 6.7.2.2 shows how the Text-to-speech is done.

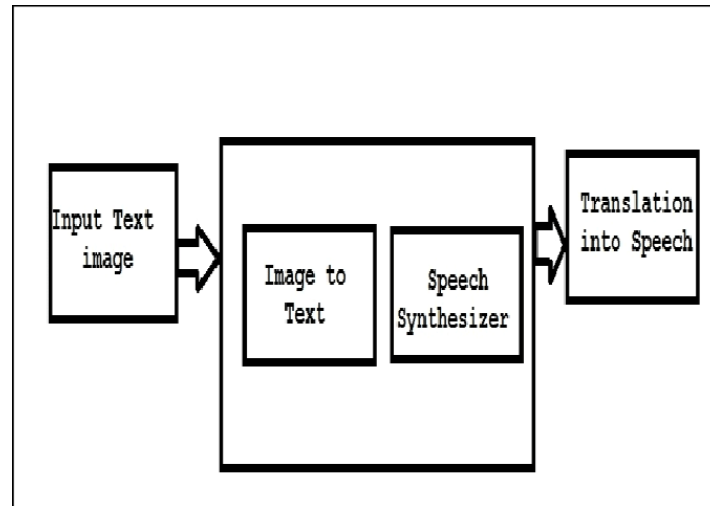


Fig 6.7.3.2 text-to-speech working

2. Speech to Text

The below Fig 6.7.2.3 shows how is Speech to Text conversion done.

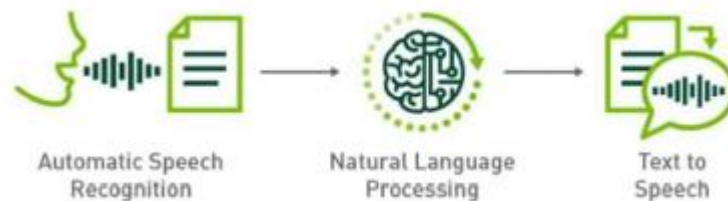


Fig 6.7.3.3 Working of Speech-to-text

6.7.4 Evaluation

1. *Voice Recognition Metric*: Metric that shows how accurate the system recognition the voice command and translates into text.
2. *Text to Speech Metric*: Metric which shows us how good the System can convert Text to Speech.

6.8 OS Agents: Using AI Techniques in the Operating System Environment

6.8.1 Introduction

The paper discusses a novel approach to operating system interfaces through OS Agents, which leverage AI techniques to enhance command execution. Unlike conventional interfaces that rely on static, user-defined commands or shell scripts, OS Agents allow users to specify high-level goals. The system dynamically synthesizes command sequences, handles execution, and adapts to environmental conditions using AI-driven planning and learning mechanisms. This goal-oriented paradigm is designed to simplify complex tasks for naive users while retaining flexibility and efficiency in execution.

6.8.2 Characteristics and Implementation

1. Goal oriented Design

Implementation: By decomposing high level user task to sub level tasks and give it to goal manager and communicate it with planner.

2. Adaptive planning

Implementation: By making a effective planner that executes commands and if any failed commands takes action accordingly.

6.8.3 Features

1. Error Handling
2. Customizability
3. Extensibility
4. Distributed Support

6.8.4 Evaluation

We can evaluate the system by implementing the prototype into UNIX environment and introducing the system to modest computational overhead, scalability, and robustness in adapting to diverse problems.

CHAPTER 7: OVERVIEW OF DATASETS

7.1 DESCRIPTION

The organized collection of data with which the developers work to achieve their goal. The rows of the dataset represent the number of points and the column represents the features of the dataset.

Datasets can be stored in many forms such as CSV, Excel, Json and zip file for large datasets such as image datasets.

Why to use Dataset?

- Datasets are used to train and test AI models.
- To analyze the trend and to gain information from data.
- They provide raw data for systems to learn pattern and make predictions.

Data VS Dataset:

- data includes numerical data, categorical data and so on.
- data alone cannot be used for making predictions.
- Where dataset is a collection of data, it contains data specific to its category.

Availability of dataset:

There are large number of available online platform which serves as a source of dataset some of them are

- Kaggle
- Google Dataset Search
- Data.Gov
- And our system data.

Data attributes/Features:

Data attributes refers to the specific characteristic that describes an object in any given dataset

Types of dataset features are as follows:

- Numerical features: this contains the dataset in a continuous form
- Categorical features: these are non-numerical dataset with or without any order of the data
- Binary features: the dataset is either classified as 0 or 1, yes or no.
- Text features: it consists of unstructured text data

CHAPTER 8: CONCLUSION OF CAPSTONE PROJECT PHASE – 1

Phase 1 of the AVIOS project has provided a strong foundation for integrating AI-powered fault detection and self-healing capabilities into an operating system. During this phase, we have:

1. Defined the Scope:

Identified critical gaps in traditional fault management systems, focusing on the need for AI-driven, autonomous fault detection and recovery.

2. Explored Existing Research:

Conducted a thorough literature survey to understand current approaches and their limitations, such as the lack of real-time adaptability and scalability.

3. Outlined Clear Objectives:

Established a roadmap for building robust fault detection mechanisms and efficient self-healing systems that address memory and security challenges.

4. Prepared for Implementation:

Designed the initial architecture for integrating fault detection and self-healing modules with the Linux kernel, ensuring feasibility and scalability.

Looking Ahead

In Phase 2, we will focus on developing high-level designs and initial implementations of these core features. By addressing challenges such as real-time efficiency and scalability, the AVIOS project aims to redefine system reliability and security through AI innovation.

This updated approach ensures our work aligns with the evolving demands of modern computing environments, emphasizing fault tolerance and seamless user experience.

CHAPTER 9: PLAN OF WORK FOR CAPSTONE PROJECT PHASE - 2

Phase 1 Deliverables:

- Background study
- Formulation of problem statement
- Literature survey

Suggestions from review 1:

Instead of working on 4 goals, work on any one of the feature and integrate with Linux kernel at max not necessary for the full kernel design because of the complexity.

Suggestion Assessment:

So as panel has suggested to go with one feature we are planning of moving forward in implementing the features like Smart Security Assistance and Intelligent Dynamic Allocation making system efficient.

Plan Of Work:

- Literature survey
- Using available open resources like IEEE Xplore for the survey.
- Collect the information on the most recent research in the field of AI in OS
- Collecting the drawbacks of the survey.
- Understanding the challenges in implementing the feature and working on it
- Collecting the dataset relevant to our problem statement from the websites such as Kaggle and from our systems as well.

REFERENCES/BIBLIOGRAPHY

- [1] "The Role of Artificial Intelligence in Modern Operating Systems," ITEJ, July 2024.
- [2] "AIOS: Towards Intelligent Operating Systems with Advanced Management Capabilities," AIOS Research Paper.
- [3] "Advances in Distributed Operating Systems for AI Integration," Distributed Systems Review.
- [4] Literature on Collaborative Filtering Algorithms and Deep Learning Techniques for Application Usage Analysis.
- [5] Research on Distributed Knowledge Networks and Parallel Search Algorithms for AI Applications.
- [6] "Historical and Modern Trends in Operating System Architectures," System Design Review.
- [7] Rahman, R., Apriliyani, A.W. and Ibrahim, S.N.A., 2024. Development of an Operating System Supporting Intelligent Predictions and Recommendations. *ITEJ (Information Technology Engineering Journals)*, 9(1), pp.1-14.
- [8] Sumathi, S., Nivetha, N. and Jovita, J.P., 2023, August. Artificially Intelligent Operating System with Sapi5 Voice Recognition Engine. In *2023 9th International Conference on Smart Computing and Communications (ICSCC)* (pp. 477-481). IEEE.
- [9] Mandrakov, E.S., Vasiliev, V.A. and Dudina, D.A., 2021, September. Application of expert systems in quality management. In *2021 International Conference on Quality Management, Transport and Information Security, Information Technologies (IT&QM&IS)* (pp. 516-518). IEEE.
- [10] Montezanti, D., De Giusti, A., Naiouf, M., Villamayor, J., Rexachs, D. and Luque, E., 2017, July. A methodology for soft errors detection and automatic recovery. In *2017 International Conference on High Performance Computing & Simulation (HPCS)* (pp. 434-441). IEEE.
- [11] Ramezani, R. and Sedaghat, Y., 2013. An overview of fault tolerance techniques for real-time operating systems. *ICCKE 2013*, pp.1-6.
- [12] Amri, M.Z. and Suryani, V., 2024, July. Android API Malware Detection Based on Permission Using Machine Learning and Deep Learning Methods. In *2024 International Conference on Data Science and Its Applications (ICoDSA)* (pp. 146-149). IEEE.
- [13] Vishwakarma, S., 2021, May. AI based OS. In *National Conference on "Unprecedented and Advanced Concepts of Computer Vision" NCUACC* (Vol. 11, No. 12).
- [14] Etzioni, O., Levy, H.M., Segal, R.B. and Thekkath, C.A., 1993. Os agents: Using ai techniques in the operating system environment. *Seattle, Washington, Tech. Rep. UW-CSE-93-04-04, University of Washington*.

APPENDIX A DEFINITIONS, ACRONYMS, AND ABBREVIATIONS

TERM	DEFINITION
AI	Artificial Intelligence: A branch of computer science focused on creating systems that perform tasks requiring human intelligence.
AIOS	Artificial Intelligence Operating System: An OS enhanced with AI features for adaptability, automation, and intelligent decision-making.
OS	Operating System: Software that manages hardware, software, and user interactions.
IoT	Internet of Things: A network of interconnected devices that communicate and exchange data.
ML	Machine Learning: A subset of AI that uses algorithms to learn from data and make predictions.
NLP	Natural Language Processing: A field of AI focused on the interaction between computers and human languages.
GPU	Graphics Processing Unit: Specialized hardware for rendering images and running complex computations.
Expert System	A computer system that mimics human expertise to solve problems and automate decision-making.
Fuzzy Logic	A method of reasoning that mimics human decision-making, handling imprecise inputs effectively.
Neural Network	An AI model inspired by the human brain to identify patterns and predict outcomes.
Pattern Recognition	A technology enabling systems to learn and adapt based on user behavior or data patterns.

ACRONYM	EXPANSION
AIOS	Artificial Intelligence Operating System
RAM	Random Access Memory
CPU	Central Processing Unit
HCI	Human-computer Interaction

ABBREVIATION	DEFINATION
AVIOS	Adaptable, Vigilant, Intelligent Operating System
AI	Artificial Intelligence
OS	Operating System
AIOS	Artificial Intelligence Operating System
IDS	Intrusion Detection System
wrt	With Respect To
FL	Fuzzy Logic
psutil	Python System and process utilities
Scikit-learn	A machine learning library in Python

LIST OF FIGURES

<i>Fig no.</i>	<i>Title</i>	<i>Page no.</i>
<i>Fig 6.2.4.1</i>	<i>Shows multiple scenarios testing of the system</i>	<i>15</i>
<i>Fig 6.5.2.1</i>	<i>Flow chart of the System.</i>	<i>18</i>
<i>Fig 6.5.4.1</i>	<i>Performance metric</i>	<i>19</i>
<i>Fig 6.6.2.1</i>	<i>Expert System Architecture</i>	<i>20</i>
<i>Fig 6.7.2.1</i>	<i>speech recognition engine working</i>	<i>22</i>
<i>Fig 6.7.3.2</i>	<i>text-to-speech working</i>	<i>23</i>
<i>Fig 6.7.3.3</i>	<i>Working of Speech-to-text</i>	<i>23</i>